

21-DARS

Iteratorlar

`iter()`, `next()`, iterator protokoli va o'z iteratoringizni yaratish

1. Iterator nima?

Iterator — bu elementlarni birma-bir qaytaruvchi obyekt. List, tuple, string — bularning hammasi iteratsiya qilinadigan (iterable) obyektlardir. `iter()` ulardan iterator yasaydi, `next()` esa keyingi elementni oladi:

```
mevalar = ('olma', 'banan', 'gilos')
it = iter(mevalar)
print(next(it)) # olma
print(next(it)) # banan
print(next(it)) # gilos
```

`next()` har safar keyingi elementni beradi.

2. String ham iterator

```
matn = 'Python'
it = iter(matn)
print(next(it)) # P
print(next(it)) # y
```

Har bir harf alohida olinadi.

3. for tsikli aslida iterator ishlatadi

`for` tsikli ortda `iter()` va `next()` ni o'zi chaqiradi. Elementlar tugaganda `StopIteration` yuz beradi va tsikl to'xtaydi:

```
for meva in ('olma', 'banan'):
    print(meva)
```

`for` — iteratorning qulay shakli.

4. O'z iteratoringizni yaratish

Sinfga `__iter__` va `__next__` metodlarini qo'shib, o'z iteratoringizni yasaysiz:

```
class Sanagich:
    def __iter__(self):
        self.a = 1
        return self
    def __next__(self):
        if self.a <= 5:
            x = self.a
            self.a += 1
            return x
        else:
            raise StopIteration

for son in Sanagich():
    print(son)
```

1 dan 5 gacha chiqaradi.

Eslatma: StopIteration tsiklni to'xtatadi. Aks holda iterator cheksiz davom etadi.

Xulosa

- `iter()` iterator yasaydi, `next()` keyingi elementni beradi.
- List, tuple, string — iterable obyektlar.
- `for` ichida iter/next avtomatik ishlaydi.
- `__iter__` va `__next__` bilan o'z iteratoringiz.